EstateFlow — Architecture Baseline v1.0 (Revision 4, FINAL)

Locked implementation baseline • Desktop property and rental management

JDK 25 LTS • JavaFX 25 • MySQL 8.4 LTS • Maven • Plain JDBC

Status: LOCKED — single source of truth for implementation

Scope: Desktop application (JavaFX), single user, local MySQL

Change policy: See Section 14. No casual changes to stack, schema, layer rules, or class responsibilities.

Revision 4: Final pre-implementation review corrections based on a technical review of Revision 3. The platform remains JDK 25 + JavaFX 25; the database is now MySQL 8.4 LTS. Corrections clarify layer dependencies, locking order, affected-row contracts, report DTOs, clock/config injection, pinned build dependencies, safe integration-test isolation, and reactivation rules. The seven-table domain and overall layered architecture remain intact. Revision 4 supersedes Revision 3 and is the single source of truth.

1. Purpose and Guiding Principles

EstateFlow manages properties, owners, tenants, rental agreements, rent collection, and maintenance for a small landlord or property manager.

Principles, in priority order:

1. **A finished, tested application beats a large unfinished one.**
2. **Understanding over magic.** Plain JDBC, no ORM, no DI framework. Every line should be explainable in a viva.
3. **Proper OOP and layering.** Each class has one clear job; dependencies point one way. Abstractions exist only where a real requirement needs them.
4. **One definition for every fact.** Derive values (occupancy, charge status, expiry) instead of storing copies that can go stale. Where a definition must exist in both SQL and Java, it is written down once (Section 6.4) and tested for agreement.
5. **Testable by construction.** Business rules are pure Java, separated from JavaFX and SQL.
6. **Failures are handled once, at the right layer, with the cause preserved.** (Section 7)

2. Technology Stack (Locked)

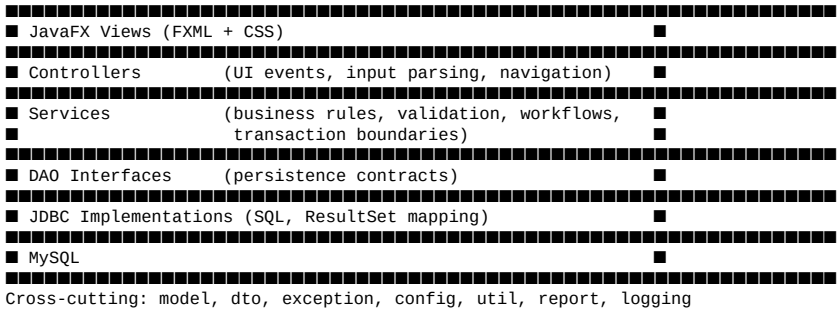
Concern	Decision	Notes
Language	Java 25 (LTS)	Use the standard, non-preview Java 25 language and platform features. Pin Java 25 as the compiler release in `pom.xml`; do not enable preview features for v1.0.
GUI	JavaFX 25 + FXML + JavaFX CSS	Align JavaFX to major version 25 with JDK 25. Load via Maven dependencies and `javafx-maven-plugin`; pin a compatible JavaFX 25 patch version in `pom.xml`.
Database	MySQL 8.4 LTS, InnoDB, `utf8mb4`	Use the MySQL 8.4 LTS series. MySQL 8.0 reached end of life in April 2026. CHECK constraints and stored generated columns are supported.
Connectivity	Plain JDBC, `mysql-connector-j`	Pin one exact Connector/J version in `pom.xml` that supports MySQL 8.4 and JDK 25. Use `PreparedStatement` only. No connection pool in v1 (see D-03).
Data access	DAO pattern (interfaces + JDBC implementations)	
Build	Maven (single module)	
Testing	JUnit 5, Mockito, JaCoCo; AssertJ optional	Pin exact dependency/plugin versions in `pom.xml`. JaCoCo must be at least 0.8.14 for official Java 25 class-file support; use a current compatible release. Service tests use Mockito. DAO tests run against a real MySQL test schema. No H2.
Logging	SLF4J API + Logback	Console + rolling file.
Version control	Git/GitHub	Section 12.

Build configuration: Pin exact versions for JavaFX 25.x, JUnit 5, Mockito 5.x, MySQL Connector/J, JaCoCo, `maven-compiler-plugin`, `maven-surefire-plugin`, and `javafx-maven-plugin`. Do not use `LATEST`, version ranges, or floating versions. Keep the exact resolved versions recorded in `pom.xml`; document the chosen toolchain in the README. Use the classpath build: do not create `module-info.java` in v1.0.

Initial compatibility pins for Phase 0 (2026-10-10): JavaFX `25.0.4`; JUnit Jupiter `5.14.4`; Mockito `5.24.0`; MySQL Connector/J `26.7.0` (supports MySQL Server 8.4+); JaCoCo `0.8.15`; Maven Compiler Plugin `3.16.0`; Maven Surefire Plugin `3.6.0`; JavaFX Maven Plugin `0.0.8`. Pin compatible exact versions for SLF4J and Logback as well. If a selected artifact is unavailable from Maven Central or proves incompatible in the initial build, choose the latest stable compatible release, update this table and changelog before proceeding, and commit the exact version in `pom.xml`. Do not use Java preview features.

Explicitly not used in v1.0: Spring/Spring Boot, Hibernate/JPA, Lombok, Flyway/Liquibase, Guice/Dagger, Testcontainers, HikariCP, Apache Commons CSV, PDF libraries, Java serialization. Reasons are in Section 13.

3. Architecture Overview



3.1 Layer dependency rules (enforced in code review)

Layer	May depend on	Must NOT depend on
Controller	Service, model, dto, exception, util (UI helpers)	DAO, `java.sql.*`, SQL
Service	DAO **interfaces** (including `TransactionManager`), model, dto, exception, `Clock`	`javafx.*`, `dao.jdbc.*`, `db.*`, `java.sql.*`
DAO interface	model, dto, exception	everything else
JDBC DAO	DAO interface, model, dto, exception, `DatabaseManager`/`ConnectionHandle`, `java.sql.*`	Services, `javafx.*`, business rules
db package	`config`, exception, `java.sql.*`, and the dao interfaces `TransactionManager` / `TransactionalWork`	Services, controllers, `dao.jdbc`
Model	`java.*` only	everything else

Consequences: services can be tested with mocked DAOs; DAOs can be tested without a GUI; models stay plain Java.

3.2 Composition root (no DI framework, no static singletons)

`AppContext` is the only place that constructs `AppConfig`, `DatabaseManager`, the transaction manager, DAOs and services, and wires them together by constructor injection. Controllers receive services through an `FXMLLoader` controller factory. Nothing is a Singleton-pattern class; each shared object is created once in `AppContext` and passed where needed.

3.3 Threading rule

All database work triggered from the UI runs on a background thread through a small `UiTask` helper (wrapping `javafx.concurrent.Task`; it accepts a `Callable`, so checked exceptions flow to its failure callback). One service call runs entirely on one worker thread. UI updates happen on the JavaFX Application Thread. `UiTask` is introduced in Phase 4 together with the first screen.

3.4 Connection ownership and transaction rules

Three collaborators, three clearly separate jobs:

Component	Owns	Never does
`DatabaseManager` (db)	Creating physical connections from `AppConfig`; a thread-bound "transaction connection" slot (package-private `bind`/`unbind`); the public method `acquire()`	Commit, roll back, or decide when a transaction starts
`JdbcTransactionManager` (db, implements the `TransactionManager` interface from dao)	Beginning, committing, and rolling back a transaction; binding/unbinding its connection on the current thread	Run SQL
JDBC DAOs (`dao.jdbc`)	Running SQL on whatever connection `acquire()` hands them	Open raw connections, call `commit`, `rollback`, `setAutoCommit`, or `close` on a `Connection`, or let a `Connection` escape their method

The mechanism: `ConnectionHandle`**.** `DatabaseManager.acquire()` returns a `ConnectionHandle` (an `AutoCloseable` wrapper):

- If a transaction is bound to the current thread, the handle **borrows** that connection. Its `close()` does nothing.
- Otherwise the handle **owns** a fresh connection (auto-commit on). Its `close()` really closes it.

DAOs always follow one pattern, identical whether or not a transaction is active:

```
try (ConnectionHandle h = databaseManager.acquire();
     PreparedStatement ps = h.connection().prepareStatement(SQL)) {
    ...
} catch (SQLException e) {
    throw translate(e, "find owner by id");
}
```

Because closing a borrowed handle is a no-op, a DAO cannot accidentally close a transaction's connection, and no DAO needs to know whether it is inside a transaction.

Rules

1. Services start transactions only through `TransactionManager.inTransaction(TransactionalWork<T>)`. Services never see a `Connection` or `ConnectionHandle`.
2. A transaction is begun, committed or rolled back on the same thread; its connection is never passed to another thread.
3. **Any** exception leaving the work (checked, unchecked, or `Error`) causes rollback; the original exception is rethrown unchanged. If the rollback itself fails, that failure is attached with `addSuppressed` (not logged separately).
4. A failed commit surfaces as `DataAccessException` with the `SQLException` as cause.
5. Nested `inTransaction` calls **join** the outer transaction; only the outermost call commits or rolls back. Work inside a transaction must never catch and swallow an exception, because the transaction would still commit.
6. Cleanup (unbind, close) happens in a `finally` block.
7. Single operations outside a transaction auto-commit on their own owned connection.
8. Row locks (`SELECT ... FOR UPDATE`) are used in exactly two places where a read-then-write check protects money or overlap integrity: `PropertyDao.findByIdForUpdate` (agreement creation) and `RentChargeDao.findByIdForUpdate` (recording a payment). They are only valid inside a transaction. The locking read must be the first SQL/data-access statement in that transaction; do not perform a prior plain consistent read that can establish a stale REPEATABLE READ snapshot. After the lock is acquired, run `findOverlapping` or the charge-balance/`amountPaid` query. Do not calculate `amountPaid` from a subquery whose rows are assumed locked by an outer `FOR UPDATE`; any required locking must be explicit. Other validation reads that do not need transactional state should happen before `inTransaction`, or after the lock as appropriate. EstateFlow supports one running application instance at a time, but duplicate Save actions can still create worker-thread races, so the locks remain necessary.
9. Every `try-with-resources` on a `ConnectionHandle` is mandatory; forgetting it leaks an owned connection (code-review item).
10. For each of the two locked flows, Phase 3 integration tests must include a two-thread race test coordinated with latches: overlapping agreement creation for the same property, and two payment attempts competing for the same charge balance. Disable the relevant Save action while a UI task is running as a user-interface backstop, not as a replacement for database locking.

Transactional operations in v1.0: create agreement (lock property, check overlap, insert agreement, insert charges); terminate agreement (cancel charges, update agreement); record payment (lock charge, validate, insert payment); void payment is a single update.

4. Project Structure

Base package: `com.estateflow` (confirm once before Phase 0 ends; do not rename afterward).

```
estateflow/
■■■ pom.xml
■■■ README.md
■■■ .gitignore
■■■ docs/
■   ■■■ ESTATEFLOW_ARCHITECTURE_BASELINE_v1.0.md
■   ■■■ er-diagram.png                (added in Phase 3)
■   ■■■ change-requests/              (ACR-001.md, ...)
■■■ db/
■   ■■■ schema.sql
■   ■■■ seed.sql                      (demo data)
■   ■■■ test-schema-setup.md
■■■ src/
    ■■■ main/
    ■   ■■■ java/com/estateflow/
    ■   ■   ■■■ Launcher.java                (non-JavaFX main; calls EstateFlowApp)
```

```
■   ■   ■■■ EstateFlowApp.java           (extends Application)
■   ■   ■■■ config/      AppConfig, AppContext
■   ■   ■■■ model/       Person, Owner, Tenant, Property, RentalAgreement,
■   ■   ■                RentCharge, Payment, MaintenanceRequest
■   ■   ■■■ model/enums/ PropertyType, AgreementStatus, EffectiveAgreementStatus,
■   ■   ■                ChargeStatus, PaymentMethod, MaintenancePriority,
■   ■   ■                MaintenanceStatus
■   ■   ■■■ dto/         DashboardStats, OverdueRentRow, ExpiringLeaseRow,
■   ■   ■                PropertyOccupancyRow, PaymentReceiptData, ...  (records)
■   ■   ■■■ dao/         TransactionManager, TransactionalWork<T>,
■   ■   ■                CrudDao<T,ID>, OwnerDao, TenantDao, PropertyDao,
■   ■   ■                AgreementDao, MaintenanceDao,
■   ■   ■                RentChargeDao, PaymentDao, ReportingDao
■   ■   ■■■ dao/jdbc/    BaseJdbcDao, RowMapper<T>, SqlFragments,
■   ■   ■                Jdbc*Dao implementations
■   ■   ■■■ db/          DatabaseManager, ConnectionHandle, JdbcTransactionManager
■   ■   ■■■ service/     OwnerService, TenantService, PropertyService,
■   ■   ■                AgreementService, PaymentService, MaintenanceService,
■   ■   ■                DashboardService, ReportService, ReceiptService
■   ■   ■■■ service/rules/ RentScheduleGenerator, Validators
■   ■   ■■■ controller/  MainController, DashboardController, and one
■   ■   ■                controller per FXML screen / dialog
■   ■   ■■■ report/      CsvWriter, ReportDefinition<T>, PaymentsReport,
■   ■   ■                OverdueRentReport, OccupancyReport, MaintenanceReport,
■   ■   ■                ReceiptWriter
■   ■   ■■■ exception/   see Section 7
■   ■   ■■■ util/        UiTask, ErrorHandler, AlertUtil, SceneNavigator,
■   ■                    InputParser, DateFormats, MoneyUtil
■   ■■■ resources/
■       ■■■ fxml/        main.fxml, dashboard.fxml, owners.fxml, ...
■       ■■■ css/         estateflow.css
■       ■■■ config/      application.properties (non-secret defaults)
■       ■■■ logback.xml
■■■ test/
    ■■■ java/com/estateflow/  (mirrors main; includes PassThroughTransactionManager)
    ■■■ resources/
```

`util` is not a dumping ground. A class belongs there only if it is stateless (or purely UI plumbing), has a narrow purpose, and fits no layer. Anything with business meaning goes in `service` or `service/rules`.

5. Domain Model and OOP Design

5.1 Conventions

- Money: `BigDecimal`, scale 2, `RoundingMode.HALF_UP`. Never `double`.
- Dates: `java.time.LocalDate` / `LocalDateTime`. "Today" always comes from an injected `java.time.Clock`.
- IDs: `Long`, `null` before insert.
- Models are plain POJOs (no JavaFX properties, no JDBC types). Controllers adapt them for `TableView` through cell value factories.
- **Where validation lives:** services validate user input (through `Validators`) and throw `ValidationException` *before* building or saving a model. Models only guard against programmer errors (`Objects.requireNonNull` for required fields, `IllegalStateException` for illegal state changes). Models do not re-validate formats or ranges.
- Enums are persisted as `VARCHAR` using `name()`.
- Read-only query results (joins, aggregates) are Java `record` DTOs, not models.
- `equals`/`hashCode` on entities are based on ID when non-null; `toString` omits sensitive fields.

5.2 Entities and responsibilities

Class	Responsibility
`Person` (abstract)	Shared state of the people EstateFlow tracks: `fullName`, `phone`, `email`, `address`, `active`; `getDisplayName()`. Abstract because a bare "person" is never created. Exists for shared state and so that shared validation (`Validators.validateContact(Person)`) and shared UI code can accept either subtype.
`Owner extends Person`	A property owner. Adds no fields; it is a distinct type so an owner can never be passed where a tenant is required.
`Tenant extends Person`	Adds `idProofType`, `idProofNumber`, `emergencyContactName`, `emergencyContactPhone`.
`Property`	Belongs to one `Owner` (by `ownerId`). Descriptive data, `PropertyType`, `listedRent`, `active`. **Does not store occupancy.**

Class	Responsibility
`RentalAgreement`	Links one `Property` and one `Tenant`. Holds `startDate`, `durationMonths`, `endDate`, `monthlyRent`, `securityDeposit`, `status` (ACTIVE/TERMINATED), `terminationDate`. Owns the **effective period** logic: `getEffectiveEndDate()`, `occupies(date)`, `overlaps(from, to)`, `effectiveStatus(today, warningDays)`.
`RentCharge`	One month's rent obligation: `periodStart`, `periodEnd`, `dueDate`, `amountDue`, `cancelledAt`. Carries a read-only `amountPaid` (sum of valid payments, filled by the DAO query). Methods: `getBalance()`, `isPayable(today)`, `status(today)`.
`Payment`	A payment received against one `RentCharge`: `amount`, `paymentDate`, `PaymentMethod`, `referenceNo`, `notes`. The payment facts have no setters. Void state changes only through `markVoided(reason, at)`, which throws `IllegalStateException` if already voided. `isVoid()`; derived `receiptNumber()` = `RCP-<paymentDate year>-<id padded to 6>`; the year comes from `payment_date`, not `created_at`.
`MaintenanceRequest`	Issue raised for a property (optionally by a tenant). `MaintenancePriority`, `MaintenanceStatus`, dates, optional `cost` and `assignedTo`. Status changes go through `MaintenanceStatus.canTransitionTo(...)`.

5.3 Enums

Enum	Values	Behavior
`PropertyType`	APARTMENT, HOUSE, VILLA, SHOP, OFFICE, LAND, OTHER	display label
`AgreementStatus` (stored)	ACTIVE, TERMINATED	
`EffectiveAgreementStatus` (derived)	UPCOMING, ACTIVE, EXPIRING_SOON, EXPIRED, TERMINATED	never stored
`ChargeStatus` (derived)	PENDING, PARTIALLY_PAID, PAID, OVERDUE, CANCELLED	never stored
`PaymentMethod`	CASH, UPI, BANK_TRANSFER, CHEQUE, CARD	display label
`MaintenancePriority`	LOW, MEDIUM, HIGH, URGENT	ordering
`MaintenanceStatus`	OPEN, IN_PROGRESS, RESOLVED, CANCELLED	`canTransitionTo()` encodes the workflow

Maintenance workflow: `OPEN` → `IN_PROGRESS` → `RESOLVED`; `OPEN` or `IN_PROGRESS` → `CANCELLED`; `RESOLVED` and `CANCELLED` are terminal.

5.4 Derived values (never stored)

Value	Rule
Effective period of an agreement	`[start_date, effective_end_date]`, where `effective_end_date = termination_date` if terminated, otherwise `end_date` (BR-12)
Open agreement	`effective_end_date >= today`
Property occupied on date *d*	Some agreement has `start_date <= d <= effective_end_date`. Otherwise vacant.
Agreement effective status	If `today < start_date`: `UPCOMING`. If `today > effective_end_date`: `TERMINATED` if stored so, else `EXPIRED`. Otherwise: `EXPIRING_SOON` if `effective_end_date <= today + warningDays`, else `ACTIVE`.
Valid payment	`voided_at IS NULL`
Live charge	`cancelled_at IS NULL`
Charge balance	`amount_due − SUM(valid payments for that charge)`
Payable charge	Live and `due_date <= today` (BR-20)
Charge status	`CANCELLED` if cancelled; else `PAID` if balance = 0; else `OVERDUE` if `due_date < today`; else `PARTIALLY_PAID` if paid > 0; else `PENDING`
Outstanding balance	Sum of balances of live charges with `due_date <= today` (future charges excluded; this is exactly the set of payable charges)
Overdue	Live charge with `due_date < today` and balance > 0

This removes the need for scheduled jobs or status-sync code.

5.5 OOP concepts: where they arise from real requirements

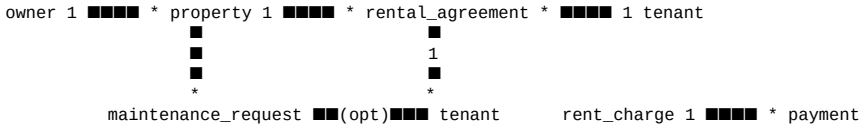
Concept	Where it appears, and why it is needed
Abstraction and inheritance	`Person` → `Owner`/`Tenant` (shared contact state and validation). `BaseJdbcDao` → every `Jdbc*Dao` (shared query helpers and the single place where `SQLException` is translated).
Encapsulation	`RentalAgreement` owns the effective-period rules; `RentCharge` owns balance/status; `Payment` exposes no setters and controls its own void transition; `MaintenanceStatus` owns its workflow. Callers ask the object instead of re-implementing the rule.
Interfaces	DAO interfaces (services depend on contracts; JDBC is swappable and mockable). `TransactionManager` (services stay JDBC-free; tests use a pass-through implementation). `ReportDefinition<T>`.
Polymorphism	Four `ReportDefinition` implementations used by one `CsvWriter`. Mocked vs JDBC DAOs behind one interface. `Validators.validateContact(Person)` accepting either subtype.
Generics	`CrudDao<T, ID>`, `ReportDefinition<T>`, `RowMapper<T>`, `TransactionalWork<T>`, `Optional<T>` returns.
Enums with behavior	`MaintenanceStatus.canTransitionTo`, display labels.
Custom exceptions and chaining	Section 7.
Collections and streams	`List`/`Map` results, grouping for dashboard and report data, `Optional` for lookups. Used where the code needs them; none are added for show.
File handling (NIO)	CSV export (`CsvWriter`) and receipts (`ReceiptWriter`).
Multithreading	`UiTask` keeps JDBC off the JavaFX thread.
Lambdas / functional interfaces	`RowMapper<T>`, `TransactionalWork<T>`, JavaFX cell factories and event handlers.
JDBC	`DatabaseManager`, `BaseJdbcDao`, `Jdbc*Dao`, `JdbcTransactionManager`.
Logging	SLF4J, with the single-logging-point policy in Section 7.

Deliberately not used: Java serialization (nothing in EstateFlow needs it; MySQL and CSV cover persistence and export). If your syllabus makes it mandatory, the one natural fit is a small `UserPreferences` class (last export folder, window size); add it in Phase 7 under a minor note in the changelog. Interfaces with a single implementation and no real second use (for example a `ReceiptGenerator` for a future PDF writer) are not created in advance.

Map your university syllabus topics against this table in Phase 0. Any uncovered topic gets a small, honest place in the design only if a real need exists.

6. Database Design (Locked Schema)

6.1 Entity relationships



- One owner has many properties. Each property has exactly one owner (co-ownership is out of scope).
- A property has many agreements over time; their effective periods may never intersect.
- An agreement has exactly one tenant and many rent charges.
- A rent charge has many payments (this is how partial payments work). Charges are cancelled, never deleted. Payments are voided, never deleted.
- A maintenance request belongs to a property and optionally references the reporting tenant.

6.2 DDL

```
CREATE DATABASE IF NOT EXISTS estateflow
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE estateflow;

CREATE TABLE owner (
  owner_id    BIGINT       NOT NULL AUTO_INCREMENT,
  full_name   VARCHAR(100) NOT NULL,
  phone       VARCHAR(20)  NOT NULL,
  email       VARCHAR(120) NULL,
  address     VARCHAR(255) NULL,
  is_active   TINYINT(1)   NOT NULL DEFAULT 1,
  created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (owner_id),
  UNIQUE KEY uq_owner_phone (phone)
) ENGINE=InnoDB;

CREATE TABLE tenant (
  tenant_id               BIGINT       NOT NULL AUTO_INCREMENT,
  full_name               VARCHAR(100) NOT NULL,
  phone                   VARCHAR(20)  NOT NULL,
  email                   VARCHAR(120) NULL,
  address                 VARCHAR(255) NULL,
  id_proof_type           VARCHAR(30)  NULL,
  id_proof_number         VARCHAR(50)  NULL,
  emergency_contact_name  VARCHAR(100) NULL,
  emergency_contact_phone VARCHAR(20)  NULL,
  is_active               TINYINT(1)   NOT NULL DEFAULT 1,
  created_at              TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (tenant_id),
  UNIQUE KEY uq_tenant_phone (phone),
  UNIQUE KEY uq_tenant_id_proof (id_proof_type, id_proof_number),
  CONSTRAINT ck_tenant_id_proof_pair CHECK (
       (id_proof_type IS NULL AND id_proof_number IS NULL)
    OR (id_proof_type IS NOT NULL AND id_proof_number IS NOT NULL))
) ENGINE=InnoDB;

CREATE TABLE property (
  property_id  BIGINT        NOT NULL AUTO_INCREMENT,
  owner_id     BIGINT        NOT NULL,
  name         VARCHAR(100)  NOT NULL,
  property_type VARCHAR(20)  NOT NULL,
  address_line VARCHAR(255)  NOT NULL,
  city         VARCHAR(60)   NOT NULL,
  state        VARCHAR(60)   NULL,
  postal_code  VARCHAR(12)   NULL,
  bedrooms     TINYINT UNSIGNED NULL,
  area_sqft    DECIMAL(9,2)  NULL,
  listed_rent  DECIMAL(12,2) NOT NULL,
  is_active    TINYINT(1)    NOT NULL DEFAULT 1,
  created_at   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (property_id),
  CONSTRAINT fk_property_owner FOREIGN KEY (owner_id)
    REFERENCES owner (owner_id) ON DELETE RESTRICT,
  CONSTRAINT ck_property_rent CHECK (listed_rent >= 0),
  INDEX idx_property_owner (owner_id)
) ENGINE=InnoDB;

CREATE TABLE rental_agreement (
  agreement_id       BIGINT        NOT NULL AUTO_INCREMENT,
  property_id        BIGINT        NOT NULL,
  tenant_id          BIGINT        NOT NULL,
  start_date         DATE          NOT NULL,
  duration_months    SMALLINT      NOT NULL,
  end_date           DATE          NOT NULL,
  monthly_rent       DECIMAL(12,2) NOT NULL,
  security_deposit   DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  status             VARCHAR(15)   NOT NULL DEFAULT 'ACTIVE',
  termination_date   DATE          NULL,
  notes              VARCHAR(500)  NULL,
  -- Single database-level definition of the agreement's effective end (BR-12).
  -- Never written by INSERT/UPDATE; MySQL maintains it.
  effective_end_date DATE GENERATED ALWAYS AS (COALESCE(termination_date, end_date)) STORED,
  created_at         TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (agreement_id),
  CONSTRAINT fk_agreement_property FOREIGN KEY (property_id)
    REFERENCES property (property_id) ON DELETE RESTRICT,
  CONSTRAINT fk_agreement_tenant FOREIGN KEY (tenant_id)
    REFERENCES tenant (tenant_id) ON DELETE RESTRICT,
  CONSTRAINT ck_agreement_dates   CHECK (end_date > start_date),
  CONSTRAINT ck_agreement_months  CHECK (duration_months BETWEEN 1 AND 120),
  CONSTRAINT ck_agreement_rent    CHECK (monthly_rent > 0),
  CONSTRAINT ck_agreement_deposit CHECK (security_deposit >= 0),
  CONSTRAINT ck_agreement_status  CHECK (status IN ('ACTIVE','TERMINATED')),
  CONSTRAINT ck_agreement_termination CHECK (
       (status = 'ACTIVE' AND termination_date IS NULL)
    OR (status = 'TERMINATED' AND termination_date IS NOT NULL
        AND termination_date >= start_date AND termination_date < end_date)),
  INDEX idx_agreement_property_period (property_id, start_date, effective_end_date),
  INDEX idx_agreement_tenant (tenant_id),
  INDEX idx_agreement_effective_end (effective_end_date)
) ENGINE=InnoDB;
```

```
CREATE TABLE rent_charge (
  charge_id     BIGINT        NOT NULL AUTO_INCREMENT,
  agreement_id  BIGINT        NOT NULL,
  period_start  DATE          NOT NULL,
  period_end    DATE          NOT NULL,
  due_date      DATE          NOT NULL,
  amount_due    DECIMAL(12,2) NOT NULL,
  cancelled_at  TIMESTAMP     NULL,
  PRIMARY KEY (charge_id),
  CONSTRAINT fk_charge_agreement FOREIGN KEY (agreement_id)
    REFERENCES rental_agreement (agreement_id) ON DELETE RESTRICT,
  CONSTRAINT uq_charge_period UNIQUE (agreement_id, period_start),
  CONSTRAINT ck_charge_amount CHECK (amount_due > 0),
  CONSTRAINT ck_charge_period CHECK (period_end > period_start),
  INDEX idx_charge_due_date (due_date)
) ENGINE=InnoDB;

CREATE TABLE payment (
  payment_id     BIGINT        NOT NULL AUTO_INCREMENT,
  charge_id      BIGINT        NOT NULL,
  amount         DECIMAL(12,2) NOT NULL,
  payment_date   DATE          NOT NULL,
  payment_method VARCHAR(20)   NOT NULL,
  reference_no   VARCHAR(60)   NULL,
  notes          VARCHAR(255)  NULL,
  voided_at      TIMESTAMP     NULL,
  void_reason    VARCHAR(255)  NULL,
  created_at     TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (payment_id),
  CONSTRAINT fk_payment_charge FOREIGN KEY (charge_id)
    REFERENCES rent_charge (charge_id) ON DELETE RESTRICT,
  CONSTRAINT ck_payment_amount CHECK (amount > 0),
  CONSTRAINT ck_payment_method CHECK (payment_method IN
    ('CASH','UPI','BANK_TRANSFER','CHEQUE','CARD')),
  CONSTRAINT ck_payment_void CHECK (
       (voided_at IS NULL AND void_reason IS NULL)
    OR (voided_at IS NOT NULL AND void_reason IS NOT NULL)),
  INDEX idx_payment_charge (charge_id),
  INDEX idx_payment_date (payment_date)
) ENGINE=InnoDB;

CREATE TABLE maintenance_request (
  request_id    BIGINT        NOT NULL AUTO_INCREMENT,
  property_id   BIGINT        NOT NULL,
  tenant_id     BIGINT        NULL,
  title         VARCHAR(120)  NOT NULL,
  description   TEXT          NULL,
  priority      VARCHAR(10)   NOT NULL DEFAULT 'MEDIUM',
  status        VARCHAR(15)   NOT NULL DEFAULT 'OPEN',
  reported_date DATE          NOT NULL,
  resolved_date DATE          NULL,
  assigned_to   VARCHAR(100)  NULL,
  cost          DECIMAL(12,2) NULL,
  created_at    TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at    TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP
                              ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (request_id),
  CONSTRAINT fk_maint_property FOREIGN KEY (property_id)
    REFERENCES property (property_id) ON DELETE RESTRICT,
  CONSTRAINT fk_maint_tenant FOREIGN KEY (tenant_id)
    REFERENCES tenant (tenant_id) ON DELETE SET NULL,
  CONSTRAINT ck_maint_priority CHECK (priority IN ('LOW','MEDIUM','HIGH','URGENT')),
  CONSTRAINT ck_maint_status   CHECK (status IN
    ('OPEN','IN_PROGRESS','RESOLVED','CANCELLED')),
  CONSTRAINT ck_maint_resolution CHECK (
       (status = 'RESOLVED' AND resolved_date IS NOT NULL AND resolved_date >= reported_date)
    OR (status <> 'RESOLVED' AND resolved_date IS NULL)),
  CONSTRAINT ck_maint_cost     CHECK (cost IS NULL OR cost >= 0),
  INDEX idx_maint_status (status),
  INDEX idx_maint_property (property_id)
) ENGINE=InnoDB;
```

6.3 Schema design rules

- Seven tables. No stored status that can be derived.
- **No hard deletes of business records.** Owners, tenants, and properties are deactivated (`is_active = 0`). Rent charges are cancelled (`cancelled_at`). Payments are voided (`voided_at`). All business FKs are `ON DELETE RESTRICT` (the only exception, `maintenance_request.tenant_id`, is `SET NULL`; since tenants are never deleted this never fires).
- Payment facts (amount, date, method, reference) are never edited. Corrections happen by voiding and re-recording (BR-25).
- INSERT statements for `rental_agreement` never list `effective_end_date`.
- `due_date` is stored separately from `period_start` (equal in v1.0) so a grace period can be added later without a schema change.
- Future tables for advanced features (`app_user`, `audit_log`) are **additive** and pre-approved in Section 11. They do not alter v1.0 tables.

6.4 Shared definitions (written once, used everywhere)

Definition	SQL form	Java form	Held in
Effective end of an agreement	`ra.effective_end_date` (generated column)	`RentalAgreement.getEffectiveEndDate()`	DB column + model method; a DAO integration test asserts they agree for active, terminated and expired rows
Valid payment	`p.voided_at IS NULL`	`!payment.isVoid()`	`SqlFragments.VALID_PAYMENT`
Live charge	`rc.cancelled_at IS NULL`	`charge.getCancelledAt() == null`	`SqlFragments.LIVE_CHARGE`

Table aliases are fixed by convention: `ra` (agreement), `rc` (rent charge), `p` (payment). Every query that lists, sums, or counts money or agreements must use these fragments instead of re-typing the condition.

7. Exceptions and Error Handling

7.1 Hierarchy

```
Exception
■■■ EstateFlowException (checked; message is always safe to show the user)
    ■■■ ValidationException        (bad or missing input; optional field name)
    ■■■ BusinessRuleException      (rule violated; carries rule id, e.g. "BR-21")
    ■■■ EntityNotFoundException    (ID not found, e.g. record changed by someone else)
    ■■■ DuplicateEntityException   (unique value already exists)
    ■■■ ReportExportException      (file output failed; cause = IOException)

RuntimeException
■■■ DataAccessException            (unchecked; infrastructure failure; cause = SQLException;
■   ■                               message includes operation, SQLState, vendor code)
■   ■■■ DuplicateKeyException      (unique-constraint violation, MySQL error 1062;
■                                   best-effort constraint name)
■■■ ConfigurationException         (unchecked; missing or invalid configuration at startup)
```

Why checked vs unchecked: business failures such as "payment exceeds balance" are *expected and recoverable*; the compiler should force every caller (ultimately a controller) to deal with them, so they are checked. Database and configuration failures are *infrastructure problems* that no caller can fix locally; making them checked would clutter every signature, so they are unchecked and handled centrally. Programmer errors (`NullPointerException`, `IllegalStateException`) are never caught to be hidden.

All `EstateFlowException` constructors accept an optional cause. `TransactionalWork<T>` declares `throws EstateFlowException`.

7.2 Who does what (translation by layer)

Layer	Responsibility	Never
JDBC DAO (`BaseJdbcDao.translate`)	Convert `SQLException` into `DataAccessException` (or `DuplicateKeyException` for error 1062). Always pass the original as cause and name the operation. This is the only place SQL error codes are inspected.	Log; show messages; contain business rules
Service	Validate input and enforce rules; throw `EstateFlowException` subtypes with specific user-readable messages. Catch `DuplicateKeyException` and rethrow as `DuplicateEntityException` with a friendly message, passing the original as cause. Let other `DataAccessExceptions` pass through untouched.	Log exceptions it rethrows; wrap a `DataAccessException` in another one; catch `Exception` broadly; swallow exceptions
`TransactionManager`	Roll back on any failure; rethrow the original; `addSuppressed` on rollback failure	Log; change the exception type
`UiTask`	Run work off the UI thread; deliver the exception (checked or not) to the failure callback on the UI thread	Interpret exceptions
Controller	Parse input (via `InputParser`, which throws `ValidationException`); pass failures to `ErrorHandler`	Show raw exception text; log

Layer	Responsibility	Never
`ErrorHandler`	The one place that turns an exception into a dialog and, when unexpected, a log entry (7.3)	

7.3 Messages shown and logging (single logging point)

Exception	Shown to the user	Logged
`ValidationException`	Its message; focus the named field when present	No
`BusinessRuleException`	Its message	No
`EntityNotFoundException`	Its message, and the list refreshes	No
`DuplicateEntityException`	Its message	No
`ReportExportException`	Its message (includes file path and reason)	WARN, once, with stack trace
`DataAccessException`	Fixed text: "The database operation failed. Check that MySQL is running. Details were written to the log." Never the SQL or technical message.	ERROR, once, full cause chain and suppressed exceptions
`ConfigurationException` (startup)	Fatal dialog, then exit	ERROR, once
Any other `RuntimeException` / `Error`	"Unexpected error. Details were written to the log."	ERROR, once, with stack trace

Logging rules

1. **Log where an exception is handled, not where it is thrown or wrapped.** DAOs, services, and the transaction manager do not log exceptions. Expected business failures are shown to the user and not logged.
2. `log.error(...)` appears only in `ErrorHandler` and in the startup path of `EstateFlowApp`. This is grep-checkable and part of the Definition of Done.
3. Services log successful business events at INFO (for example "Agreement 12 created for property 4"), without personal data such as phone numbers.
4. A global uncaught-exception handler (default thread handler, plus the FX thread) routes anything that escapes to `ErrorHandler`, so nothing is silently lost.

Exception hygiene rules

- Always chain the cause. Never call `printStackTrace()`. No empty `catch` blocks.
- Never catch `Exception`/`Throwable` except in `ErrorHandler` and `JdbcTransactionManager` (which rethrows).
- Use `try-with-resources` for every `ConnectionHandle`, `Statement`, `ResultSet`, and file stream; use `finally` only where no `AutoCloseable` fits (transaction unbind).
- Messages state what happened and, where possible, what to do, with concrete values: *"Payment of ■5,000.00 exceeds the remaining balance of ■3,200.00 for the period starting 01 Oct 2026."*
- Do not catch an exception inside a lambda just to hide a checked type; validate first, then process.

8. Business Rules (Authoritative List)

These rule IDs are referenced by tests, Javadoc, and commit messages.

Parties and properties

- **BR-01** Owner and tenant phone numbers are unique within their table. Normalize phone input before storage and uniqueness checks by trimming surrounding whitespace and removing formatting spaces, hyphens, and parentheses; preserve a leading `+` if supplied. Tenant ID proof type and number must either both be absent or both be present, and the pair is unique when provided.
- **BR-02** An owner cannot be deactivated while they own an active property. An inactive owner may be reactivated at any time.

- **BR-03** A tenant cannot be deactivated while they have an **open** agreement (`effective_end_date >= today`). An inactive tenant may be reactivated at any time.
- **BR-04** A property cannot be deactivated while it has an **open** agreement. A property may be reactivated only if its owner is active.
- **BR-05** Only active owners can receive new properties; only active tenants and properties can be used in new agreements.

Agreements

- **BR-10** An agreement is created from: property, tenant, start date, duration in months (1–120), monthly rent (> 0), security deposit (≥ 0). The start date may be in the past (to record an existing lease) or the future. Charges for past periods are generated as due, and historical payments are recorded afterwards in the normal way.
- **BR-11** `end_date = start_date + duration_months − 1 day`.
- **BR-12 Effective period.** An agreement occupies its property from `start_date` to `effective_end_date` inclusive, where `effective_end_date` is `termination_date` if the agreement is terminated and `end_date` otherwise. This is the only definition used for occupancy, overlap, deactivation, expiry listing, and effective status. Stored `status` is not used in date logic.
- **BR-13 No overlap.** Two agreements for the same property must not have intersecting effective periods (inclusive). The check runs inside the creation transaction after locking the property row, using `AgreementDao.findOverlapping`. Back-to-back agreements are allowed (new `start_date` = previous `effective_end_date` + 1 day).
- **BR-14** On creation, one `RentCharge` per month is generated in the **same transaction**: period *i* starts at `start_date.plusMonths(i)` (computed from the start date each time to avoid month-end drift), ends the day before period *i+1* starts, and is due on its `period_start`. `amount_due = monthly_rent`.
- **BR-15 Termination.** Allowed only when the agreement is `ACTIVE` and `today <= end_date` (an expired agreement cannot be terminated). `termination_date` must satisfy `start_date <= termination_date < end_date`; it may be in the past (late record-keeping) or the future (notice period). The operation is rejected if any **valid payment** exists on a charge with `period_start > termination_date`. Otherwise, in one transaction: `status = TERMINATED`, `termination_date` is set, and all live charges with `period_start > termination_date` are **cancelled** (`cancelled_at`). The period that contains the termination date stays fully chargeable (no proration, D-07).
- **BR-16** Agreement terms (rent, dates) are not editable after creation. Only `notes` may change. Mistakes are handled by terminating and re-creating.
- **BR-17 Occupancy.** A property is occupied on date *d* if some agreement occupies it on *d* (BR-12); otherwise it is vacant. A terminated agreement therefore frees the property the day after its termination date.

Rent and payments

- **BR-20 Payable charges.** A payment can only be recorded against a live charge whose `due_date <= today`. There are no advance payments against future charges in v1.0; a period becomes payable on the day it starts. Payable charges are exactly the charges counted in "outstanding balance".
- **BR-21** Payment amount > 0, at most 2 decimal places, and not more than the charge's remaining balance. The balance is read with `RentChargeDao.findByIdForUpdate` inside the recording transaction, so the check and the insert are atomic.
- **BR-22** Payment date cannot be in the future.
- **BR-23** Partial payments are allowed; a charge may have many payments.
- **BR-24** Payments can be recorded against payable charges of expired or terminated agreements (to settle arrears).
- **BR-25 Incorrect payments.** Payments are never edited or deleted. A wrong payment is **voided** with a mandatory reason (1–255 characters): `voided_at` and `void_reason` are set, the record stays in history, and the payment is excluded from every balance and total. The correct payment is then recorded as a new payment. A payment cannot be voided twice. Voiding is allowed on any valid payment, including on terminated or expired agreements. A receipt cannot be issued for a voided payment. Voided payments appear in the ledger and payments report with status VOID.
- **BR-26** Derived values follow Section 5.4 exactly. "Today" comes from the injected `Clock`.
- **BR-27** Lease-expiry warning window is configurable (`lease.expiry.warning.days`, default 30).

Maintenance

- **BR-30** Requests can only be opened for active properties; a tenant, if given, must have (or have had) an agreement on that property. `reported_date` cannot be in the future.

- **BR-31** Status changes must satisfy `MaintenanceStatus.canTransitionTo`. `resolved_date` is required exactly when status is `RESOLVED`, must not be before `reported_date`, and is `NULL` for all other statuses; the schema enforces this invariant as well as the service.
- **BR-32** `cost` may only be set when status is `IN_PROGRESS` or `RESOLVED`.

Clock/configuration rule (applies to every service and DAO): `AppContext` reads `lease.expiry.warning.days` (default 30) and passes it to each relevant service as an `int` constructor argument; services do not depend on `config`. Services receive an injected `Clock` and derive business "today" from it. Every SQL query whose result depends on today/date receives the relevant date as a bound parameter; business-date logic must never use `CURDATE()` or `NOW()`. Database-managed metadata timestamps (`created_at` defaults and `updated_at` auto-update) may use `CURRENT_TIMESTAMP`. Configure JDBC/MySQL timestamp handling to UTC consistently and test round-tripping `voided_at` and `cancelled_at`.

9. Class Responsibilities by Layer

9.1 DAO layer

`CrudDao<T, ID>`: `T insert(T)`, `int update(T)` (returns the number of matched rows), `Optional<T> findById(ID)`, `List<T> findAll()`. Set Connector/J `useAffectedRows=false` explicitly and test update-count semantics so a matched row with unchanged values is not misreported as missing. Services treat a zero-row update as a stale/missing record and raise the appropriate business exception. It is implemented only by DAOs whose entities genuinely support all four operations: `OwnerDao`, `TenantDao`, `PropertyDao`, `AgreementDao`, `MaintenanceDao`. `RentChargeDao`, `PaymentDao`, and `ReportingDao` are standalone interfaces, because payments are immutable and charges are generated in bulk. There is no delete operation anywhere; deactivation, cancellation, and voiding are updates.

DAO	Extra operations (beyond CRUD, where applicable)
`OwnerDao`	`findByPhone`, `search(text)`
`TenantDao`	`findByPhone`, `findByIdProof`, `search(text)`
`PropertyDao`	`findByOwner`, `search(text)`, `findVacantOn(date)`, `findByIdForUpdate`, `countActiveByOwner(ownerId)`
`AgreementDao`	`findByProperty`, `findByTenant`, `findOverlapping(propertyId, from, to)`, `findEndingBetween(from, to)`, `existsOpenForTenant(tenantId, today)`, `existsOpenForProperty(propertyId, today)` — all use `effective_end_date`
`RentChargeDao`	`insertAll(List)` (batch), `findByAgreement` (with `amountPaid`), `findById` (with `amountPaid`), `findByIdForUpdate`, `int cancelAfter(agreementId, date, at)` returning matched rows
`PaymentDao`	`insert`, `findById`, `findByCharge`, `findByDateRange` (includes voided, flagged), `int markVoided(paymentId, reason, at)`, `existsValidForChargesAfter(agreementId, date)`. `markVoided` updates only with `WHERE payment_id = ? AND voided_at IS NULL` and returns the matched-row count; zero means missing/already voided and is translated to a business exception.
`MaintenanceDao`	`findByProperty`, `findByStatus`, `countOpenByPriority`
`ReportingDao`	Read-only aggregate and joined-row queries returning DTO records: dashboard counts, overdue rows, occupancy rows, monthly collections, payment-report rows, and maintenance-report rows (all via `SqlFragments` where applicable)

Rules for all JDBC DAOs

- Extend `BaseJdbcDao`, which provides small helpers (`queryList`, `queryOne`, `executeUpdate`, `insertReturningKey`) and `translate(SQLException, operation)`. Concrete DAOs supply SQL, parameter binding, and a `RowMapper<T>` (a functional interface that may throw `SQLException`).
- `PreparedStatement` with bound parameters only.
- Obtain connections only through `databaseManager.acquire()` in try-with-resources (Section 3.4). Never call `commit`, `rollback`, `setAutoCommit`, or `close` on a `Connection`.
- No business logic, no logging of exceptions (Section 7).
- Use `SqlFragments` for the valid-payment, live-charge, and effective-end definitions.
- List screens and payment/maintenance reports use join DTO records (for example `AgreementListRow`, `PaymentReportRow`, `MaintenanceReportRow`) returned by DAO queries. Do not perform N+1 per-row lookups in services. `ReportingDao` owns the joined row queries needed by `PaymentsReport` and `MaintenanceReport`; `PaymentDao.findByDateRange` remains a payment-entity query. Do not duplicate the same report query in both DAOs.

9.2 Infrastructure

Class	Responsibility
`AppConfig`	Loads `application.properties` (defaults) and external `db.properties` (credentials, **never committed**; `db.properties.example` is committed). Throws `ConfigurationException` on missing/invalid values. Integration tests use a separate uncommitted `db-test.properties`, selected explicitly for tests.
`DatabaseManager`	Creates physical connections; holds the thread-bound transaction connection; exposes `acquire()` (Section 3.4).
`ConnectionHandle`	`AutoCloseable` wrapper; `connection()` accessor; `close()` is a no-op for borrowed connections and a real close for owned ones.
`TransactionManager` (interface, dao)	`<T> T inTransaction(TransactionalWork<T> work) throws EstateFlowException`. A `void` overload is provided for convenience.
`JdbcTransactionManager` (db)	Implements the transaction rules in Section 3.4.
`AppContext`	Composition root (Section 3.2).

9.3 Service layer

Service	Responsibility
`OwnerService`, `TenantService`, `PropertyService`	CRUD workflows, validation, uniqueness and deactivation rules (BR-01–05), search
`AgreementService`	Create (BR-10–14, transactional), terminate (BR-15, transactional), list, ending-soon queries, effective status
`PaymentService`	Record payment (BR-20–24, transactional), void payment (BR-25), list charges per agreement, balances, overdue and outstanding queries
`MaintenanceService`	Open/update requests, enforce workflow (BR-30–32), filters
`DashboardService`	Assemble `DashboardStats` from `ReportingDao`
`ReportService`	Fetches report data and passes it with a `ReportDefinition` to `CsvWriter`
`ReceiptService`	Builds `PaymentReceiptData` for a valid payment (BR-25) and calls `ReceiptWriter`
`RentScheduleGenerator` (rules)	Pure function: agreement terms → list of `RentCharge`. No I/O.
`Validators` (rules)	Reusable checks (non-blank, phone/email format, money scale, date order, `validateContact(Person)`) throwing `ValidationException`

Services never format strings for the UI beyond what exports need; presentation formatting belongs to controllers/`DateFormats`.

9.4 Controller layer

One FXML + one controller per screen/dialog:

Screen	Controller
Shell with sidebar and content area	`MainController` (uses `SceneNavigator`)
Dashboard	`DashboardController`
Owners (list + form dialog)	`OwnerListController`, `OwnerFormController`
Tenants (list + form dialog)	`TenantListController`, `TenantFormController`
Properties (list + form + detail)	`PropertyListController`, `PropertyFormController`, `PropertyDetailController`
Agreements (list + create + detail/terminate)	`AgreementListController`, `AgreementFormController`, `AgreementDetailController`
Rent & payments (charges per agreement, record payment dialog, void payment dialog; voided payments shown greyed)	`RentLedgerController`, `PaymentFormController`, `VoidPaymentController`
Overdue rent / expiring leases	`OverdueRentController`, `ExpiringLeasesController`
Maintenance (list + form)	`MaintenanceListController`, `MaintenanceFormController`
Reports / exports	`ReportsController`

Controllers: parse and pre-check input (`InputParser`), call one service method through `UiTask`, update the view, pass failures to `ErrorHandler`. No SQL, no business rules, no direct DAO access, no logging of exceptions.

9.5 Reports and files

- `ReportDefinition<T>`: `String fileName()`, `List<String> headers()`, `List<String> toRow(T item)`. Implementations: `PaymentsReport` (date range; includes a Status column VALID/VOID, totals count valid payments only), `OverdueRentReport`, `OccupancyReport`, `MaintenanceReport`.
- `CsvWriter`: `<T> void write(Path, ReportDefinition<T>, List<T>)`. RFC 4180 escaping (quotes, commas, newlines), UTF-8 with BOM for Excel compatibility. Wraps `IOException` in `ReportExportException`. Hand-written and unit-tested.
- `ReceiptWriter`: formatted `.txt` receipt for a valid payment. Shows receipt number, date, tenant, property, rent period, amount paid, method, reference, and the current balance of that rent period at the time of printing.
- Output location is chosen by the user through `FileChooser`; never hard-coded.

9.6 Dashboard (v1.0 metrics)

Total / occupied / vacant **active** properties and occupancy % (denominator is active properties only; BR-17); agreements occupying today; leases ending within the warning window; rent collected this month (valid payments only); total outstanding balance; number of overdue charges; open maintenance requests (by priority).

10. Implementation Phases

Each phase ends with working, committed, tested code and a git tag/milestone. Do not start a phase until the previous one meets its exit criteria.

Keep-it-small rules (apply throughout):

1. Build one class and its test at a time; commit after each step.
2. No JDBC before Phase 3 and no JavaFX before Phase 4. Each step is runnable with JUnit alone until then.
3. Implement only rules listed in Section 8 and only features listed in Section 9.
4. If a step takes more than about two days, split it.
5. Advanced features (Phase 8) are not started until `v1.0` is tagged.

Phase 0 — Foundations (≈ 2–3 days)

- Maven project, Git repo, `.gitignore`, README skeleton, folder structure, this baseline in `/docs`.
- Confirm base package; map syllabus topics against Section 5.5.
- MySQL 8.4 LTS installed; `schema.sql` runs cleanly against `estateflow`; `estateflow_test` schema created; `db.properties.example` created.
- Pin exact versions for all direct dependencies and Maven plugins in `pom.xml`; include JaCoCo (at least 0.8.14; current compatible release preferred).
- Add one trivial JUnit test and one small Mockito mock test; generate a JaCoCo report successfully. Verify JavaFX dependencies resolve at build time; defer an actual JavaFX window launch smoke test until Phase 4, when the application shell exists.
- **Exit:** project builds under JDK 25, unit tests pass, JaCoCo report generates, and schema loads on MySQL 8.4.

Phase 1 — Core Java and OOP domain (≈ 1–2 weeks)

Build order, each with tests:

1. Enums, including `MaintenanceStatus.canTransitionTo`.
2. Exception hierarchy (Section 7.1): tests for messages, cause retention, rule id and field name.
3. `Person`, `Owner`, `Tenant`, `Property`.
4. `RentalAgreement`: effective period, `occupies`, `overlaps`, `effectiveStatus`.
5. `RentCharge` and `Payment`: balance, `isPayable`, status, `markVoided`.

6. `MaintenanceRequest`.
7. `MoneyUtil`, `Validators`.

- No database, no GUI.
- **Exit:** unit tests cover all derived-value logic (Section 5.4) and validators; models have sensible constructors, `equals`/`hashCode`/`toString`.

Phase 2 — Business logic with mocked DAOs (≈ 1–2 weeks)

Build order, one slice at a time:

- **2a** DAO interfaces, `TransactionManager` interface, test-side `PassThroughTransactionManager`; lock the affected-row return contracts for `CrudDao.update`, `PaymentDao.markVoided`, and `RentChargeDao.cancelAfter`; `OwnerService`, `TenantService`, `PropertyService` (BR-01–05).
- **2b** `RentScheduleGenerator`; `AgreementService` create/terminate (BR-10–17). Inject `warningDays` via constructor; ensure property locking is the first data-access statement inside agreement-creation transaction.
- **2c** `PaymentService` record/void and balance queries (BR-20–27). Ensure charge locking is the first data-access statement inside the payment-recording transaction; use conditional void updates and affected-row results.
- **2d** `MaintenanceService` (BR-30–32).
- Tests use Mockito for DAOs and a fixed `Clock`. No in-memory DAO implementations are written.
- **Exit:** every business rule has at least one passing test (positive and negative); services contain no SQL or JavaFX imports.

Phase 3 — Database and JDBC (≈ 2 weeks)

Build order:

- **3a** `AppConfig`, `DatabaseManager`, `ConnectionHandle`; schema loads; connectivity test. Add `TestDatabase` for integration-test cleanup: use only the explicit `db-test.properties`, refuse to run unless the selected schema name ends with `_test`, truncate all seven tables in a controlled order with foreign-key checks restored in `finally`, and never fall back to production/development credentials. Mark DB tests with JUnit `@Tag("integration")`; ordinary `mvn test` in Phases 1–2 excludes that tag, and a documented Maven profile/command runs them in Phase 3.
- **3b** `JdbcOwnerDao` written longhand (try-with-resources, manual `SQLException` handling) plus its integration test, so the JDBC pattern is understood before it is abstracted.
- **3c** Extract `BaseJdbcDao`, `RowMapper`, and `translate` from the working DAO (tests stay green); then `TenantDao`, `PropertyDao`.
- **3d** `JdbcTransactionManager`; tests for: borrowed handle not closed inside a transaction, owned handle closed outside one, nested join, rollback on checked and unchecked exceptions, suppressed rollback failure.
- **3e** `AgreementDao`, `RentChargeDao`, `PaymentDao`, `MaintenanceDao`, `SqlFragments`; test that Java and SQL effective-end definitions agree; test that duplicate-key errors yield `DuplicateKeyException` with the `SQLException` as cause.
- **3f** `ReportingDao`, `seed.sql`, `AppContext` wiring; service integration tests with real DAOs for the key flows (create agreement → pay → void → terminate; failure mid-creation leaves no partial charges). Include the two-thread latch-coordinated race tests required by Section 3.4 rule 10. ER diagram added to `/docs`.
- **Exit:** all flows pass against `estateflow_test`; no DAO calls `commit`/`rollback`/`close` on a `Connection` directly.

Phase 4 — JavaFX shell and master-data screens (≈ 2 weeks)

- **4a** `EstateFlowApp`, `Launcher`, controller factory, `SceneNavigator`, `AlertUtil`, `UiTask`, `ErrorHandler`, global exception handler, base CSS.
- **4b** Owners screen end to end (list, search, add, edit, deactivate) as the template.
- **4c** Tenants, then **4d** Properties, reusing the pattern.
- **Exit:** full master-data CRUD from the UI with validation messages; DB calls off the UI thread; every failure path reaches `ErrorHandler`.

Phase 5 — Business workflow screens (≈ 2–3 weeks)

- **5a** Agreements: create with generated schedule preview; terminate.
- **5b** Rent ledger, record payment (partial payments, balance display), void payment.

- **5c** Overdue rent and expiring leases views.
- **5d** Maintenance requests with the status workflow.
- **Exit:** an end-to-end scenario works from the UI: add owner → property → tenant → agreement → partial payment → void a wrong payment and re-record → overdue visible → terminate → maintenance request lifecycle.

Phase 6 — Dashboard, exports, receipts (≈ 1 week)

- `DashboardService` + screen; `CsvWriter` with four `ReportDefinitionS`; `ReceiptWriter`; `FileChooser` integration.
- **Exit:** all reports open correctly in Excel; receipt generated from the ledger screen; voided payments flagged correctly.

Phase 7 — Hardening and release v1.0 (≈ 1 week)

- Logging review (grep check: `log.error` only in `ErrorHandler`/startup), empty/error states, keyboard and layout polish, CSS cleanup.
- Test pass: ≥ 70% line coverage on `service`, `service/rules`, `model`, `report` (guideline, not a vanity target).
- README with setup steps, screenshots, ER diagram, architecture summary; seed demo data; tag `v1.0`.
- **Exit:** Definition of Done (Section 15) is met.

Phase 8 — Advanced features (post-v1.0, each under change control)

Order of value: (1) authentication and role-based access (`app_user`, password hashing with bcrypt or PBKDF2), (2) audit history (`audit_log`), (3) packaging with `jpackage` (installer), (4) PDF receipts (introduce a `ReceiptGenerator` interface at that point, when a second implementation exists), (5) Flyway for schema migrations, (6) optional Docker Compose file for MySQL, (7) `UserPreferences` if required by the syllabus.

11. Pre-approved Future Schema Additions (Phase 8 only)

These are designed now so v1.0 is not disturbed later; they are **not** part of the v1.0 schema.

```
-- app_user: authentication and roles
--   user_id PK, username UNIQUE, password_hash, role ('ADMIN','MANAGER','VIEWER'),
--   is_active, created_at, last_login_at
-- audit_log: audit history
--   audit_id PK, user_id FK NULL, entity_type, entity_id, action,
--   old_value JSON NULL, new_value JSON NULL, created_at
```

(Payment voiding is no longer deferred: it is part of v1.0 via `payment.voided_at` / `void_reason`.)

12. Engineering Practices

Git

- `main` is always buildable. Work on short-lived branches (`feature/agreement-service`). Merge via pull request even when working alone (resume signal).
- Commit style: `type(scope): message`, e.g. `feat(payment): reject overpayment (BR-21)`.
- Milestone/tag per phase: `phase-1-complete` … `v1.0`.
- `.gitignore`: `target/`, `.idea/`, `*.iml`, `logs/`, `db.properties`, `db-test.properties`.

Testing

Level	Scope	Tools
Unit	Models, validators, exceptions, `RentScheduleGenerator`, `CsvWriter`, enums	JUnit 5
Service	Business rules with mocked DAOs, `PassThroughTransactionManager`, and a fixed `Clock`	JUnit 5 + Mockito

Level	Scope	Tools
DAO / transaction integration	Each `Jdbc*Dao`, `JdbcTransactionManager`, and key service flows against MySQL `estateflow_test`; isolated via `TestDatabase` and tagged `integration`	JUnit 5
UI	Manual test checklist (documented). No TestFX in v1.0.	

Test naming: `method_condition_expectedResult`. Every bug fix gets a regression test.

Code standards

- Constructor injection everywhere; no static access to services.
- No `System.out.println`; use SLF4J.
- No magic strings for statuses; shared SQL conditions only through `SqlFragments`.
- Methods short and single-purpose; Javadoc on public service methods (cite the BR ID).

13. Key Decisions and Rationale

ID	Decision	Reason	Rejected alternative
D-01	Layered architecture with DAO interfaces	Matches syllabus, enables mocking DAOs	Direct SQL in controllers
D-02	Plain JDBC, no ORM	Syllabus requires JDBC; learning value; fully explainable	Hibernate/JPA, Spring Data
D-03	No connection pool in v1	Single-user desktop app; local connections are cheap; less to explain. A pool can later sit behind `DatabaseManager`.	HikariCP
D-04	`TransactionManager` interface in `dao`, JDBC implementation in `db`	Services stay JDBC-free; tests use a pass-through implementation; keeps `Connection` out of service signatures	Passing `Connection` to every DAO method; services depending on `db` classes
D-05	`rent_charge` ledger table	Makes partial payments, overdue, and per-month balances simple and accurate	Computing dues by formula from dates
D-06	Derive occupancy and statuses	No stale data, no scheduled jobs	Stored `property.status` and `charge.status`
D-07	Agreement defined by start date + duration in months; no proration	Removes due-day and proration edge cases	Free-form end dates with a due-day column
D-08	Payments allocated to exactly one charge	Simple, explainable, no allocation table	Many-to-many allocation
D-09	Separate `owner`/`tenant` tables, shared `Person` in Java	Different attributes, simpler SQL; inheritance only for genuinely shared state and validation	Single `person` table; JPA inheritance
D-10	`PropertyType` enum, not subclasses	Types differ in data, not behavior	`Apartment`/`House` hierarchy
D-11	Soft deactivation, `RESTRICT` FKs, cancel/void instead of delete	Preserves financial history	Cascading or hard deletes
D-12	Injected `Clock`	Deterministic tests for overdue/expiry logic	`LocalDate.now()` in services
D-13	Real MySQL for DAO tests	H2 in MySQL mode differs in subtle ways	H2 / Testcontainers
D-14	Manual composition root, no static singletons	Visible wiring, testable, no framework to learn	Guice, Spring, Singleton classes
D-15	Hand-written `CsvWriter` driven by `ReportDefinition<T>`	Four real report types need different columns; small, testable, no dependency	Apache Commons CSV; a `ReportExporter` interface with one implementation
D-16	Concrete `ReceiptWriter`; no `ReceiptGenerator` interface in v1.0	One implementation, no current second use; the interface is introduced if/when PDF arrives	Interface "for the future"
D-17	SQL scripts instead of Flyway	Fewer moving parts; schema is locked anyway	Flyway/Liquibase in v1.0
D-18	SLF4J + Logback	Industry standard; resume-relevant	`java.util.logging`
D-19	Business logic before JDBC/GUI; testing continuous; slice-by-slice build order	Rules validated early and cheaply; each step is small	Original order (JDBC → JavaFX → business → testing)

ID	Decision	Reason	Rejected alternative
D-20	Models are plain POJOs, not JavaFX properties	Reusable, testable, no UI dependency	`StringProperty` fields in models
D-21	`ConnectionHandle` (borrowed vs owned) behind `DatabaseManager.acquire()`	DAOs cannot close a transaction's connection and need not know about transactions	DAOs checking transaction state; passing `Connection` explicitly
D-22	Effective period (`effective_end_date` generated column + one Java method)	One definition for occupancy, overlap, expiry, deactivation; fixes terminated-agreement inconsistencies	Using stored `status` in some queries and dates in others
D-23	Charges are cancelled, not deleted	Consistent with "no hard deletes"; works with FK history including voided payments	Deleting unpaid future charges
D-24	No advance payments; payable = due	Payable set equals outstanding set; no orphan balances after termination	Allowing prepayment with credit tracking
D-25	Voiding replaces editing/deleting payments (in v1.0)	Reliable audit trail and a real correction path	Append-only with no correction; editable payments
D-26	Checked business exceptions, unchecked infrastructure exceptions	Compiler forces handling of expected failures; infrastructure errors handled centrally	All unchecked; all checked
D-27	Log once, in `ErrorHandler`; DAO/service/transaction layers do not log exceptions	No duplicate log entries; expected business errors are not noise	Logging at every layer
D-28	Mockito for service tests; no in-memory DAO implementations	Fewer classes to write; tests read like rules; flow tests run against real JDBC in Phase 3	Eight hand-written fake DAOs
D-29	`CrudDao<T,ID>` only where full CRUD exists; `BaseJdbcDao` for shared JDBC plumbing	Generics and inheritance where they remove real duplication, without forcing `update` onto immutable payments	One generic DAO for everything
D-30	Single running instance; row locks only for overlap and overpayment checks	Keeps concurrency simple but the two money/overlap checks correct	Full multi-user concurrency design
D-31	Services validate; models guard only programmer errors	One place for user-facing validation; no duplicated checks	Validation in both constructors and services
D-32	MySQL 8.4 LTS	Supported LTS baseline for new implementation; avoids beginning on the now-EOL MySQL 8.0 line	MySQL 8.0
D-33	Explicit row-count contracts and conditional voiding	Services can detect stale/missing updates and cannot overwrite an already-voided payment	Silent void/update success with zero rows
D-34	Dedicated guarded integration-test schema	Real-MySQL tests remain representative without risking development data	Tests using whichever database config happens to be active

14. Change Control (Architecture Lock)

Locked (requires a change request): technology stack (Section 2); layer rules (3.1); connection and transaction rules (3.4); database schema (Section 6); exception hierarchy and handling policy (Section 7); business rules (Section 8); class responsibilities (Section 9); package structure at the top level (Section 4).

Free to change without a request: private methods and internal helpers, FXML layout and CSS, validation message wording, additional tests, additional non-public helper classes inside an existing package, performance tuning that does not alter schema or behavior.

Change request process

1. Write `docs/change-requests/ACR-NNN.md` containing: problem, why the baseline cannot handle it, proposed change, affected classes/tables/rules, effort and risk, and alternatives considered.
2. Approve explicitly, then update this document: bump the version (v1.1, v1.2 …) and add a changelog entry.
3. Implement after the baseline is updated, never before.

Additive changes already listed in Section 11 still follow steps 1–3 but are considered pre-reviewed.

Out of scope for v1.0 (not a change request; simply not built): multiple tenants per agreement, co-owners, late fees and grace periods, rent escalation and renewals, advance/credit payments, security-deposit refunds, utilities billing, document uploads, notifications/email, multi-currency, multi-user concurrent access, cloud sync, online payment-gateway processing and webhook verification. Payment methods such as UPI/card are recorded as externally completed payments; v1.0 does not initiate or verify a live transaction.

15. Definition of Done for v1.0

- All features in Section 9 work through the UI against MySQL.
- Every business rule BR-01–BR-32 has at least one automated test.
- DAO, transaction, and key service-flow integration tests pass against `estateflow_test`; the test harness refuses non-`_test` schemas.
- Race tests prove agreement overlap prevention and payment overpayment prevention under two concurrent worker threads.
- JaCoCo reports coverage; line coverage is at least 70% for `service`, `service/rules`, `model`, and `report` (guideline in Phase 7).
- No layer-rule violations (no DAO/SQL in controllers; no `javafx.*`, `db.*`, or `java.sql.*` in services; `db` may depend only on the specific `dao` transaction interfaces as stated in Section 3.1).
- No database calls on the JavaFX Application Thread.
- `log.error` appears only in `ErrorHandler` and the `EstateFlowApp` startup path; no `printStackTrace`, no empty `catch`.
- No DAO calls `commit`, `rollback`, `setAutoCommit`, or `close` on a `Connection`.
- Every `DataAccessException` retains its `SQLException` cause; the UI never shows technical database text.
- Clean clone → documented setup → running app with seed data in under 15 minutes.
- README includes screenshots, ER diagram, architecture diagram, and run/test instructions.
- Git history is clean, tagged `v1.0`.

16. Changelog

Version	Date	Change
v1.0	2026-10-10	Initial locked baseline
v1.0 Rev 2	2026-10-10	Final pre-implementation refinements (no code existed yet):
		R1 Transaction safety: `ConnectionHandle` (borrowed vs owned), `DatabaseManager.acquire()`, `TransactionManager` interface + `JdbcTransactionManager`, explicit rules (Section 3.4); row locks limited to two checks
		R2 Agreement consistency: single effective-period definition (BR-12), `effective_end_date` generated column, termination check constraint, one overlap/occupancy/expiry/deactivation logic (BR-03/04/13/15/17), new `UPCOMING` effective status, `findOverlapping`/`findEndingBetween`/`existsOpen*` DAO methods
		R3 Payment reliability: no advance payments (BR-20); locked overpayment check (BR-21); payment voiding moved into v1.0 (`voided_at`, `void_reason`, BR-25); charge cancellation instead of deletion (`cancelled_at`); shared SQL fragments for valid payment / live charge
		R4 Exceptions: checked business exceptions vs unchecked infrastructure exceptions; `DuplicateKeyException`, `ConfigurationException`; layer-by-layer translation table; single logging point (`ErrorHandler`); `InputParser`; message and hygiene rules
		R5 OOP quality: removed `Person.getRoleLabel()`, singleton classes, `ReportExporter`/`ReceiptGenerator` interfaces; added `ReportDefinition<T>` and `BaseJdbcDao`/`RowMapper<T>`; `CrudDao` limited to full-CRUD entities; validation placement defined; serialization explicitly not forced
		R6 Complexity: slice-by-slice build order per phase, keep-it-small rules, Mockito instead of in-memory DAOs, hand-written first DAO before extracting the base class
v1.0 Rev 3	2026-10-10	Pre-implementation platform alignment: JDK 21 → JDK 25 LTS; JavaFX aligned to version 25.

Version	Date	Change
v1.0 Rev 4	2026-10-10	Final pre-implementation review: clarified `db` → transaction-interface dependency; added first-read locking order and race tests; defined affected-row contracts and conditional payment voiding; specified joined report/list DTOs; injected `warningDays` and bound dates in SQL; moved to MySQL 8.4 LTS; required exact dependency/plugin pins and JaCoCo; added guarded integration-test isolation; defined reactivation; tightened tenant ID-proof, maintenance date, dashboard, receipt-year, timestamp-zone, and classpath-build rules; added a database check for maintenance resolution-date consistency. No redesign of the domain or overall layers.